A technical reference

K-Line / ISO 9141 OEM Bus & ECU Addressing

Protocol patterns, frame formats, initialization, timing, service IDs, negative-response codes, and OEM-specific addressing models from 1996 onward.

> *This document replaces a thinner earlier survey with a reference that can actually be implemented against. It covers the physical layer of K-line; the frame formats of ISO 9141-2 and ISO 14230-2 (KWP2000); both 5-baud slow initialization and the later fast init; the full KWP2000 service-ID and negative-response-code tables; the OBD-II / EOBD standard services and the Mode 01 PIDs most diagnostic tools actually use; and separate OEM protocol profiles for Volkswagen/Audi Group, BMW, Subaru, Mercedes-Benz, Nissan, and the remaining volume manufacturers that used K-line in their post-1996 era.*

PREPARED FOR · Jakka

PREPARED · 19 April 2026

SCOPE · OEM diagnostic K-line / ISO 9141 / ISO 14230 (KWP2000) usage, 1996 onward

SOURCES · ISO 9141-2, ISO 14230 parts 1-4, SAE J1979, SAE J1962; open OEM tool documentation (VCDS, RomRaider, EDIABAS/INPA, DataScan II)

AUDIENCE · Diagnostic-tool implementers, ECU reverse-engineers, vehicle-firmware developers

REVISION · 2.0 · compiled from primary protocol standards and open documentation

CONTENTS

Contents

SECTION 1

Scope, conventions, and what this document does not cover.

This document describes the use of K-line (ISO 9141-2) and its KWP2000 successor (ISO 14230-1 through ISO 14230-4) as deployed by vehicle manufacturers from model year 1996 onward. It covers: the electrical layer; the frame formats used on the wire; the two initialization procedures that bring an ECU out of bus-idle into a diagnostic session; the timing parameters that govern bus turn-around; the KWP2000 application-layer service catalogue; the OBD-II / EOBD standard services mandated for emissions diagnostics; and the manufacturer-specific dialects that sit on top of the same physical and data-link layers with their own addressing conventions, services, and module maps.

What this document is not

This is not an ISO standards substitute. ISO 9141-2, ISO 14230 (parts 1–4), and SAE J1979 / J1962 are the authoritative sources for the behaviours catalogued here and should be consulted for any production-grade implementation. This document synthesises those standards with publicly-documented OEM practice to give an implementer enough context to read the standards with intent and make correct design decisions about which protocol profile to instantiate for a given vehicle.

It is also not a complete enhanced-diagnostic reference for any single OEM. The volume of manufacturer-specific detail (every routine, every local identifier, every coding block, every actuator test) runs to tens of thousands of pages per brand and lives in factory workshop systems. This reference covers the scaffolding that lets an implementer reach those details: protocol, init, addressing, session management, and the common service surface.

Conventions

ITEM	MEANING
Hex notation	Single bytes are shown as `0xNN`. Byte sequences on the wire are shown space-separated without 0x prefixes: `C2 33 F1 01 00 E7`.
Direction	Tester-to-ECU frames are labelled **tester** → **ECU**. ECU-to-tester frames are labelled **ECU** → **tester**. Where an OEM-specific name differs from ISO-standard terminology, both are given.
Timing	All timing values are in milliseconds unless otherwise noted. Tolerance is shown where the standard defines one.
Addressing	"Physical addressing" means a message carries a target ECU byte. "Functional addressing" means the target byte selects a function group (e.g. `0x33` = OBD emissions-related ECUs). "Logical controller number" is a tool-facing module number (e.g. VAG controller 01) that does not appear on the wire.
Reserved values	`0x33` is reserved by ISO for OBD functional broadcast. `0xF1` is the ISO-conventional tester source byte but many OEM scan tools use other values (e.g. `0xF0` on Subaru SSM).

Cross-OEM portability warning

The only values safe to hard-code across every OEM are the ones defined by ISO or SAE: K-line on pin 7, optional L-line on pin 15, 10400 bps bit rate, 5-baud init address `0x33` for OBD, tester source `0xF1` for ISO

14230-4 OBD, and the Mode 01 PID table. Everything else — the target bytes of enhanced ECUs, the meaning of local identifiers, the services supported, the session-timing parameters — is OEM and platform specific. The rest of this document is organised around that distinction.

SECTION 2

Physical layer — J1962 connector, K-line, L-line, adapters.

J1962 (16-pin OBD-II) connector

SAE J1962 defines the 16-pin trapezoidal connector mandated on OBD-II-compliant vehicles sold in the US from 1996 (and in the EU from 2001 for petrol, 2004 for diesel, as EOBD). The connector exposes up to five diagnostic buses on fixed pins; the remaining pins are manufacturer-specific and were used variously for proprietary buses, wake-up lines, and pre-CAN vehicle networks.

PIN	ASSIGNMENT	NOTES
1	Manufacturer-specific	Commonly used for GM single-wire CAN (GMLAN), Ford MS-CAN, or vendor wake-up
2	J1850 bus+	SAE J1850 (GM Class 2 VPW, Ford SCP PWM)
3	Manufacturer-specific	Often a second K-line or a CAN-B / LIN link on some Euro OEMs
4	Chassis ground	Connected to vehicle chassis
5	Signal ground	Logic reference. Often (but not always) bonded to pin 4 internally
6	CAN-H (ISO 15765-4)	High-speed CAN, 500 kbps, mandatory on OBD-II from MY2008 US
7	**K-line** (ISO 9141-2, ISO 14230-4)	Primary diagnostic K-line. This is the subject of this reference.
8	Manufacturer-specific	BMW DS2 K-line on many E36/E39/E46 cars (bridge to pin 7 required by aftermarket tools)
9	Manufacturer-specific	Frequently a secondary bus; e.g. GM Class 2 diagnostic
10	J1850 bus-	Used only on Ford SCP (PWM). Not used on VPW.
11	Manufacturer-specific	Often unused on modern vehicles
12	Manufacturer-specific	Ditto
13	Manufacturer-specific	Ditto
14	CAN-L (ISO 15765-4)	Low-speed CAN line of the OBD CAN pair
15	**L-line** (ISO 9141-2, optional)	Initialization-only on many older ECUs; driven in parallel with K during 5-baud init
16	Battery positive	Permanent +12V / +24V, 4A max draw. Present even with ignition off.

K-line electrical characteristics

K-line is a single-wire, half-duplex, open-collector (or equivalent open-drain) bus. In idle the line is pulled high to battery potential through a resistor (typically 510Ω at the ECU, sometimes 1 kΩ). Any node can drive the line low to transmit. Bus-level collisions are not arbitrated at the physical layer: if two nodes transmit simultaneously the frames collide and detection falls to the message layer (header, length, and checksum mismatches). In practice the protocol is tester-initiated and ECUs respond in turn, so collisions are rare outside bus faults.

PARAMETER	VALUE
Idle voltage	V_{BAT} (nominally 12 V on light vehicles, 24 V on heavy)

Dominant (active-low) voltage	$< 0.2 \cdot V_{BAT}$ at the receiver; typically < 1.5 V at the driver
Receiver threshold	Typically mid-rail with 1–2 V hysteresis; exact values ECU-dependent
Pull-up resistor	510 Ω to V_{BAT} (ECU-side). 1 kΩ on some ECUs
Driver type	Open-collector / open-drain. External high-side driver is common in tester interfaces
Bit rate, typical	10 400 bps ± 0.5% (ISO 9141-2 default, ISO 14230-4 OBD)
Bit rate, allowed	ISO 14230-1 allows up to 125 000 bps; most OEM K-line runs at 10 400 bps
UART format	8 data bits, 1 start, 1 stop, no parity (8N1), LSB-first
Max cable length	Approximately 5 m at 10 400 bps without repeaters

L-line and its modern (ir)relevance

The original ISO 9141 specification required the initialization address byte to be driven on the L-line (pin 15) while K-line was held idle; after init, L-line was unused and K-line carried all traffic. ISO 9141-2 made L-line optional, and most vehicles from about 2000 onward accept 5-baud init on K-line alone. The L-line survives in the connector standard mainly as a compatibility hook for early-1990s European ECUs. A robust tester interface drives both K and L during 5-baud init and transmits/receives on K only for data exchange.

Adapter-level quirks worth knowing

A minimal K-line adapter is a level-shifter between PC UART logic (TTL 3.3 V or 5 V) and the vehicle K-line (V_{BAT} swing, open-drain). The canonical IC is the STMicroelectronics L9637, which provides a K-line driver and receiver with correct biasing and ESD protection. Most consumer USB-K-line cables (including the various VAG-COM, K+DCAN, and Subaru SSM clones) are an FTDI FT232 or Prolific PL2303 USB-UART followed by an L9637 front-end.

ADAPTER QUIRK	DETAIL
BMW DS2 on E36/E39/E46	DS2 uses K-line on J1962 pin 8 rather than pin 7. Factory tools used the 20-pin round under-hood connector, which exposed DS2 K-line directly. Aftermarket OBD-II tools for DS2 typically require pin 7 and pin 8 bridged at the OBD connector. Treat this as a cable modification, not a protocol quirk.
Mercedes 38-pin round	Pre-2000 Mercedes used a 38-pin round diagnostic connector under the hood. The 16-pin OBD-II connector (where present) exposes only the emissions ECU; non-engine modules are only reachable via the 38-pin connector. A 38-pin-to-16-pin adapter is required for most enhanced diagnostics on pre-2000 cars.
Subaru SSM1 (pre-1996)	SSM1 used a proprietary 9-pin connector under the dash and a 1953 bps bit rate (not 10 400). SSM1 is not covered in this reference; SSM2 (post-1996) is, and is accessible on the standard J1962 pin 7.
VAG KW1281 cables	KW1281 is bit-compatible with ISO 9141 at the physical layer; any standard K-line interface can carry it. The protocol peculiarity (byte-by-byte inverted echo) is a higher-layer concern. See § 8.
Cheap clones and galvanic isolation	Many low-cost cables omit galvanic isolation. On a car with a weak battery or ignition-transient issues, an unisolated cable can back-feed noise into the PC USB hub and crash the adapter. For workshop use, an opto-isolated K-line driver (e.g. L9637 behind an IL711 or equivalent) is worth the extra cost.

SECTION 3

Frame formats — ISO 9141-2 and ISO 14230-2 (KWP2000).

The two standards

There are two ISO standards that govern frames on K-line. ISO 9141-2 is the older; it defines a short-header frame intended for emissions diagnostics and is mandatory for OBD-II compliance on vehicles using K-line. ISO 14230-2 (the data-link part of KWP2000) supersedes it functionally, adds session management, variable-length headers, and a richer service set, and is used both in OBD-II mode (ISO 14230-4) and in many OEM-specific enhanced-diagnostic stacks. An ECU can, and usually does, support both; which one is used is negotiated during initialization via the keyword bytes returned by the ECU.

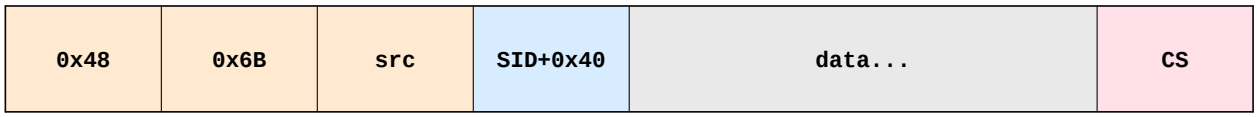
ISO 9141-2 frame

ISO 9141-2 defines a simple fixed-header frame. The header is three bytes: a format byte (always `0x68` on request, `0x48` on response), a target byte, and a source byte. The data field that follows is 1 to 7 bytes long, and the frame ends with an 8-bit sum checksum.

FRAME — ISO 9141-2 request (tester → ECU)

Legend: `0x68` = format byte (request), `0x6A` = diagnostic target, `0xF1` = tester source, SID = service identifier, CS = 8-bit sum of all preceding bytes mod 256.

FRAME — ISO 9141-2 response (ECU → tester)



The response-side format byte is `0x48`, target (`0x6B`) indicates the tester, and the source byte is the ECU's physical address. A positive response echoes the request SID with bit 6 set (SID + 0x40). A negative response uses SID `0x7F` followed by the original SID and an NRC byte.

ISO 9141-2 constrains the data field to between 1 and 7 bytes inclusive, which caps the total frame at 11 bytes. This is tight for anything beyond the basic OBD Mode 01 queries, which is why KWP2000 with its variable-length frame replaced it for enhanced diagnostics.

ISO 14230-2 (KWP2000) frame

KWP2000 defines a variable-length frame with three possible header layouts, controlled by the top two bits of the format byte and the presence or absence of a separate length byte. The bottom six bits of the format byte (L5…L0) encode the data-field length; if L5…L0 = 0, a dedicated length byte follows the addresses and carries the length (1–255). This permits frames from a single data byte up to 255.

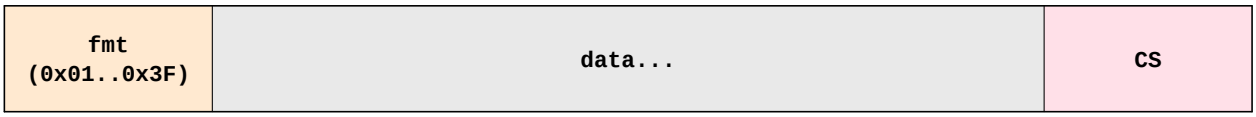
The format byte, bit by bit

BIT	MEANING	DESCRIPTION
bit 7	`A1`	Addressing mode, MSB

bit 6	`A0`	Addressing mode, LSB. A1:A0 = 00 → no address information; 01 → CARB mode (exception); 10 → physical; 11 → functional
bits 5–0	`L5…L0`	Data-field length 1–63, or 0 if a dedicated length byte follows

The three header layouts

LAYOUT 1 — format byte only, no address (rarely used for OBD)



LAYOUT 2 — 3-byte header: format + target + source, length in format byte

LAYOUT 3 — 4-byte header: format + target + source + length byte (1..255)

In practice, OBD-II over KWP2000 (ISO 14230-4) uses layout 2 with functional addressing for requests and layout 2 with physical addressing for responses. Typical format bytes are `0xC2` (functional, len=2) and `0x83` through `0x86` (physical, len=3..6). Layout 3 is used when responses exceed 63 data bytes, which is typical for VIN reads and some OEM data-by-identifier responses.

Checksum

On both ISO 9141-2 and ISO 14230-2, the checksum is the 8-bit arithmetic sum of every byte in the frame before the checksum byte, taken modulo 256. No XOR, no CRC, no polynomial — just a running 8-bit addition. Many ECUs reject frames whose checksum is zero even if arithmetically correct (some implementations treat zero as "not computed"); when constructing a frame that happens to produce a zero sum, pad or alter it rather than transmit a zero checksum byte.

Worked example — OBD Mode 01 PID 0x0C (RPM) over KWP2000

```
# Request: tester 0xF1 -> functional 0x33, service 01, PID 0x0C
C2 33 F1 01 0C 13
   \__\__\__\__\__ checksum = (0xC2+0x33+0xF1+0x01+0x0C) mod 256 = 0x13

# Positive response from engine ECU at physical address 0x10:
#   format 0x84 (physical, len=4), target 0xF1, source 0x10, service+0x40=0x41,
#   PID 0x0C, two data bytes (A=0x1A, B=0xF8), checksum
84 F1 10 41 0C 1A F8 99
   RPM = (A*256 + B)/4 = (0x1A*256 + 0xF8)/4 = 6904/4 = 1726 rpm

# Negative response (e.g. ECU busy):
83 F1 10 7F 01 21 A5
   SID 0x7F = negative, original SID 0x01, NRC 0x21 = busyRepeatRequest
```

SECTION 4

Initialization — 5-baud slow init and fast init.

An ECU on K-line sits in low-power idle until the tester wakes it. Two initialization procedures are defined. The older is the 5-baud slow init, which selects target ECU, protocol variant, and timing profile in a single roughly 2.5-second handshake. The newer is the fast init, added in ISO 14230-2, which skips the slow address byte in favour of a short line-low pulse followed by a StartCommunication request. Both remain in use; which one an ECU accepts is part of its platform-specific profile.

5-baud slow initialization

The tester transmits a single byte at 5 bits per second. At that rate, one byte frame (1 start bit + 8 data bits + 1 stop bit = 10 bits) takes exactly 2 000 ms to send. The byte carries either the OBD functional address `0x33` or an OEM-specific address selecting a particular ECU family (for example VAG uses the logical-controller number directly, so `0x01` selects the engine, `0x17` selects the instrument cluster, and so on). The ECU samples the slow bits with a timer, recognises the address, and begins responding at the negotiated bit rate.

Sequence, step by step

STEP	SIDE	WIRE CONTENT	TIMING
1	Tester	`5-baud address byte (e.g. 0x33)`	~2000 ms at 5 bps
2	ECU	`Sync byte 0x55 at 10400 bps`	after W1 = 60..300 ms
3	ECU	`Keyword byte 1 (KB1)`	after W2 = 5..20 ms
4	ECU	`Keyword byte 2 (KB2)`	after W3 = 0..20 ms
5	Tester	`Inverted KB2 (~KB2)`	within W4 = 25..50 ms
6	ECU	`Inverted address byte (~addr)`	within W5 < 300 ms

The inverted-KB2 echo from the tester and the inverted-address echo from the ECU serve as a mutual handshake confirmation. If either side sees a mismatch it should abort and return to idle; most implementations retry the full 5-baud sequence after a back-off period of at least 300 ms to let the other side time out its own state.

Keyword byte meanings

KB1 and KB2 together negotiate which protocol variant is in effect. KB1 encodes header-byte and length-byte support; KB2 encodes timing-parameter support. The exact bit packing is specified in ISO 14230-2 but most implementers only need to recognise the handful of KB pairs actually seen in the field:

KB1	KB2	VARIANT	MEANING
`0x08`	`0x08`	ISO 9141-2	Fixed 3-byte header, 1..7 data bytes, no session management
`0x94`	`0x94`	ISO 14230-4 KWP2000 (slow init)	OBD-II via KWP2000. Targets `0x33` functional, responds at the ECU's physical address.
`0xEF`	`0x8F`	ISO 14230 (fast init, StartComm)	Returned by ECU in the positive response to StartCommunication after fast init. Most common enhanced value.
`0x01`	`0x8A`	VAG KW1281	VAG proprietary block protocol with inverted-echo handshake. Used pre-KWP2000 on nearly all VAG cars.

`0x6B`	`0x8F`	ISO 14230 variant	Seen on some Bosch-supplied ECUs. Full 4-byte header supported.
`0xE9`	`0x8F`	ISO 14230 variant	Seen on some Siemens / Continental ECUs.

Fast initialization (ISO 14230-2)

Fast init replaces the 2-second slow address byte with a short line-low pulse. It is mandatory on ISO 14230-4 OBD-II implementations from the late 2000s onward and is also used in many OEM enhanced stacks because it lets the tester begin exchanging frames within about 100 ms of cable connection rather than 3 s.

Sequence, step by step

STEP	SIDE	WIRE CONTENT	TIMING
1	Tester	K-line held low	T_{iniL} = 25 ± 1 ms
2	Tester	K-line released (pulled high by bus)	until T_{WuP} = 50 ± 1 ms total
3	Tester	StartCommunication request	first bit within T_{WuP} + 50 ms
4	ECU	Positive response to StartCommunication	within P2 max

The StartCommunication request on the wire

```
# Tester -> ECU: StartCommunication (service 0x81), functional broadcast for OBD
C1 33 F1 81 66
   C1 = format byte: functional addressing (11xxxxxx), length 1
   33 = target (OBD functional)
   F1 = source (tester)
   81 = SID StartCommunication
   66 = checksum = (0xC1+0x33+0xF1+0x81) mod 256

# ECU -> Tester: positive response, physical addressed back to tester
83 F1 10 C1 EF 8F 47
   83 = format byte: physical addressing (10xxxxxx), length 3
   F1 = target (tester)
   10 = source (engine ECU physical address)
   C1 = service response (0x81 + 0x40)
   EF = key byte 1 (ISO 14230 header modes supported)
   8F = key byte 2 (ISO 14230 timing profile)
   47 = checksum
```

Why there are two init methods

The slow init exists for backward compatibility: ISO 9141-capable ECUs recognise the 5-baud address byte and respond with `0x55` followed by `0x08 0x08` to declare themselves as ISO 9141-2, while ISO 14230-capable ECUs respond with `0x94 0x94` to the same stimulus. A tester that supports both variants can thus discover which protocol the ECU speaks from the same initialization sequence. Fast init cannot do this — there is no keyword-byte handshake, only a timed pulse and a StartCommunication frame — so it is used only when the protocol is known in advance to be KWP2000.

SECTION 5

Timing parameters — P1–P4, W1–W5, tester-present cadence.

KWP2000 defines two timing-parameter families. The P-parameters govern steady-state message-to-message timing on the bus. The W-parameters govern the initialization handshake (they have already been referenced in § 4). An implementation that gets these wrong will see ECUs silently ignore requests, drop sessions on the idle timer, or corrupt responses with mid-frame gaps that exceed the inter-byte limit.

P-parameters — steady-state timing

PARAM	MEANING	MIN (MS)	MAX (MS)	NOTES
P1	Inter-byte time from ECU	0	20	Maximum gap between bytes within a single ECU-to-tester frame. Exceeding this aborts frame reception on the tester.
P2	Time from tester request to ECU response start	25	50	The default. Extended by 0x78 (response pending) to a profile-specific maximum, often 5000 ms.
P2*	Extended P2 after a 0x78 response-pending NRC	25	5000	Negotiable via service 0x83 AccessTimingParameters. Implementations should re-arm this on every 0x78 received.
P3	Idle time between end of ECU response and next tester request	55	5000	Minimum is bus turnaround; maximum is the session-idle timeout, after which the ECU drops out of its diagnostic session.
P4	Inter-byte time from tester	5	20	Gap between bytes within a tester-to-ECU frame. Most implementations transmit back-to-back (gap = 0) and rely on the 10400 bps line rate.

W-parameters — 5-baud init handshake

PARAM	MEANING	MIN (MS)	MAX (MS)	NOTES
W1	Address byte end → sync byte 0x55	60	300	Long window because the ECU samples 5-baud timing in software and may service other tasks.
W2	Sync byte end → keyword byte 1	5	20	Short; implementations can assume KB1 follows promptly.
W3	KB1 end → KB2	0	20	KB1 and KB2 are often transmitted back-to-back.
W4	KB2 end → tester inverted KB2	25	50	Tester-side timing. Missing this window is the most common cause of failed slow inits.
W5	Inverted KB2 end → ECU inverted addr	0	300	ECU finalises handshake and prepares session.

Tester-present keep-alive

Once a diagnostic session has been started, the ECU arms an inactivity timer. If the tester does not transmit any frame within P3 (typically 5 s in the default session, sometimes longer in programming sessions), the ECU drops the session and returns to its default state — and may set a DTC for "diagnostic session lost" as a side-effect. To prevent this, the tester periodically transmits service `0x3E` TesterPresent. A common cadence is once per 2 s, giving a comfortable margin below the 5 s default P3 max.

```
# Tester -> ECU: TesterPresent, no response required
82 10 F1 3E 01 02
   82 = format: physical, length 2
   10 = target (ECU)
   F1 = source (tester)
   3E = SID TesterPresent
   01 = sub-function: suppressPosRspMsgIndicationBit (don't reply)
   02 = checksum

# With sub-function 00 instead of 01, ECU returns a positive response:
# 82 F1 10 7E 00 E1
```

Timing profiles in practice

The ISO 14230-2 defaults listed above are what an OBD-II implementation is guaranteed. OEM enhanced stacks routinely use different values: VAG commonly extends P2 max to 1000 ms to accommodate long-running read operations; BMW DS2 uses significantly shorter inter-byte tolerances because the bus is point-to-point at the adapter; Subaru SSM2 has essentially no P3 ceiling because the ECU does not arm a session-idle timer. A tester that needs to span multiple OEMs should derive timing from a per-profile table rather than hard-code ISO defaults.

SECTION 6

KWP2000 application layer — sessions, services, responses.

The session concept

KWP2000 splits the ECU's diagnostic behaviour into *sessions*. A session is a mode with a defined set of enabled services. An ECU that has just powered up is in the default ("standard") session, in which it responds to OBD-style reads but not to memory writes, routine controls, or programming. To unlock richer functionality the tester issues service `0x10` StartDiagnosticSession with an appropriate sub-function byte.

SUB-FUNC	NAME	MEANING
`0x81`	`standardDiagnosticSession`	Default. OBD reads and error-memory operations.
`0x85`	`programmingSession`	Enables flash reprogramming services (0x34 RequestDownload, 0x36 TransferData, 0x37 RequestTransferExit). Bit rate often switches to a higher value.
`0x86`	`extendedDiagnosticSession`	Enables routines, actuator tests, writing by identifier, security access. The most-used OEM enhanced session.
`0x92`	`EOLProductionSession`	End-of-line / factory session. Rarely accessible to aftermarket tools. Enables coding and adaptations.
`0xA0`	`OEMSupplierSession (example)`	OEM-defined. Values above 0x80 are largely vendor-specific; consult the OEM profile.

Request and positive response

Every request carries a service identifier (SID) in the first data byte. The positive response echoes the SID with bit 6 set: SID + 0x40. So 0x10 becomes 0x50, 0x22 becomes 0x62, 0x27 becomes 0x67, and so on. The remainder of the response is service-specific.

Negative responses and NRCs

A negative response has SID `0x7F`, followed by the original request's SID, followed by a single negative-response code (NRC) byte. The complete list of NRCs is in Appendix B; the three most commonly seen in the field are `0x11` serviceNotSupported, `0x12` subFunctionNotSupported, and `0x22` conditionsNotCorrect (typically returned when the ECU is awake but in the wrong session for the requested service).

The special case: 0x78 response pending

NRC `0x78`, requestCorrectlyReceived-ResponsePending, is a temporary negative response. The ECU is telling the tester: "I got your request, I'm working on it, don't time me out." Each `0x78` resets the P2 timer, allowing the ECU to extend its response latency up to the full P2* max (often 5 s) for each occurrence. A tester must treat `0x78` specially: do not count it as an error, do not retry the request, and do not abort the session. Simply re-arm the P2 timer and keep listening.

Service catalogue, grouped

KWP2000 defines roughly thirty services. They cluster into six functional groups as shown below; Appendix A has the exhaustive list with SID values, sub-function structure, and typical use.

GROUP	SERVICES
Diagnostic management	0x10 StartDiagnosticSession, 0x11 ECUReset, 0x20 StopDiagnosticSession, 0x3E TesterPresent, 0x81 StartCommunication, 0x82 StopCommunication, 0x83 AccessTimingParameters
Data transmission	0x21 ReadDataByLocalIdentifier, 0x22 ReadDataByCommonIdentifier, 0x23 ReadMemoryByAddress, 0x2C DynamicallyDefineLocalIdentifier, 0x2E WriteDataByCommonIdentifier, 0x3B WriteDataByLocalIdentifier, 0x3D WriteMemoryByAddress
Stored data transmission	0x14 ClearDiagnosticInformation, 0x17 ReadStatusOfDTC, 0x18 ReadDTCByStatus, 0x1A ReadECUIdentification
Input / output control	0x2F InputOutputControlByCommonIdentifier, 0x30 InputOutputControlByLocalIdentifier
Remote activation	0x31 StartRoutineByLocalIdentifier, 0x32 StopRoutineByLocalIdentifier, 0x33 RequestRoutineResultsByLocalIdentifier, 0x38 StartRoutineByAddress, 0x39 StopRoutineByAddress, 0x3A RequestRoutineResultsByAddress
Upload / download	0x27 SecurityAccess, 0x34 RequestDownload, 0x35 RequestUpload, 0x36 TransferData, 0x37 RequestTransferExit

SecurityAccess: the seed-and-key dance

Before an ECU allows writes to flash, or in many cases writes to adaptation RAM, it requires unlocking via service `0x27`. The sequence is: tester requests a seed (sub-function 0x01, 0x03, 0x05, …), ECU returns a pseudorandom seed (typically 2 or 4 bytes), tester computes a key using a manufacturer-specific algorithm, tester sends the key back (sub-function 0x02, 0x04, 0x06, …), ECU returns positive response if the key matches. The key algorithm is not in any ISO standard; every OEM has one or several, and they are the point at which open tooling typically stops. Some algorithms are simple XOR permutations; others are full block ciphers with a per-ECU key schedule.

SECTION 7

OBD-II / EOBD — standard services and Mode 01 PIDs.

SAE J1979 (and its ISO counterpart ISO 15031-5) defines a set of emissions-related services that every OBD-II-compliant vehicle must support. These are the services a generic scan tool can rely on across every brand. The ten defined modes and their KWP2000 equivalents are:

MODE	PURPOSE	KWP2000 RELATIONSHIP
0x01	Show current data	ReadDataByCommonIdentifier variant
0x02	Show freeze frame data	Freeze frame captured at DTC storage
0x03	Show stored DTCs	ReadDTCByStatus, status = confirmed
0x04	Clear DTCs and stored values	ClearDiagnosticInformation
0x05	O2 sensor monitoring test results	Mostly superseded by Mode 06
0x06	On-board monitoring test results	Catalyst, EGR, evap, misfire monitors
0x07	Show pending DTCs	ReadDTCByStatus, status = pending
0x08	Control on-board system / component	Bi-directional control, e.g. evap-leak test
0x09	Vehicle information	VIN, calibration IDs, CVN, ECU name
0x0A	Permanent DTCs	Introduced MY2010; cannot be cleared by scan tool

Mode 01 PIDs

Mode 01 ("show current data") takes a second byte, the PID (parameter ID), selecting which live value to return. A small cross-manufacturer subset accounts for most tool usage; a selected list is given below and the full selected reference is in Appendix D.

PID	NAME	BYTES	FORMULA / UNIT
0x00	PIDs supported 01-20	4 bytes	Bitmask. Bit set = PID supported.
0x01	Monitor status since DTCs cleared	4 bytes	MIL status, readiness monitors
0x04	Calculated engine load	1 byte	A * 100 / 255 [%]
0x05	Engine coolant temp	1 byte	A - 40 [°C]
0x0B	Intake MAP	1 byte	A [kPa]
0x0C	Engine RPM	2 bytes	(A*256 + B) / 4 [rpm]
0x0D	Vehicle speed	1 byte	A [km/h]
0x0E	Timing advance	1 byte	(A/2) - 64 [° before TDC]
0x0F	Intake air temp	1 byte	A - 40 [°C]
0x10	MAF air flow	2 bytes	(A*256 + B) / 100 [g/s]
0x11	Throttle position	1 byte	A * 100 / 255 [%]
0x1F	Runtime since engine start	2 bytes	A*256 + B [s]

`0x2F`	Fuel tank level	1 byte	A * 100 / 255 [%]
`0x42`	Control module voltage	2 bytes	(A*256 + B) / 1000 [V]
`0x46`	Ambient air temp	1 byte	A - 40 [°C]

Mode 09 — vehicle information

Mode 09 carries a second byte selecting which information item to return. Common items are `0x02` VIN, `0x04` Calibration ID, `0x06` Calibration Verification Number, `0x08` In-use Performance Tracking, and `0x0A` ECU name. The VIN, at 17 ASCII bytes, is the canonical reason a response may exceed the ISO 9141-2 7-byte data limit and force the transaction onto KWP2000 with layout 3 (separate length byte).

Why enhanced diagnostics exist at all

The OBD-II / EOBD services listed above are sufficient for emissions compliance but are nearly useless for most workshop tasks: no ability to read individual sensor values by their real names, no actuator tests, no access to comfort or body modules, no ability to code or program modules, no access to non-emission DTCs, no freeze-frame detail beyond the emissions-required subset. Every OEM built a richer set of services on top of the KWP2000 scaffolding. Those OEM extensions are what § 8 through § 13 catalogue.

SECTION 8

OEM profile — Volkswagen Audi Group (KW1281 and KWP2000).

Volkswagen AG (VW, Audi, SEAT, Škoda, Bentley) produced the longest, most self-consistent and best-documented K-line diagnostic ecosystem of any OEM. From roughly 1990 to 2003 VAG vehicles used KW1281, a proprietary block protocol that rides on the ISO 9141 physical layer but with its own framing. From 1998 onward, newer ECUs on the same platforms migrated to KWP2000 (VAG internally called it KWP6000) while retaining the same logical-controller numbering. The controller numbers (01 Engine, 02 Transmission, 03 ABS, …) have survived onto CAN and UDS on current vehicles and remain the standard way VAG tools expose modules.

Addressing model: logical controller numbers

VAG does not expose an ISO-style physical-target byte to tools. Instead, each module is assigned a logical controller number in the range `0x01`–`0xFF`. On KW1281 the controller number is the 5-baud init address byte itself; on KWP2000 the controller number is encoded in the physical-target byte of the frame header. In both cases the user-facing number is the same, which is why a VAG workshop tool written in 1997 and one written in 2025 both let the user select "01 Engine" or "17 Instruments" from the same dropdown.

A selection of the commonly-used controller numbers appears below; the full extended list is in Appendix C.

ADDR	MODULE	DESCRIPTION
`01`	Engine	Engine control module (petrol or diesel)
`02`	Transmission	Automatic transmission control module
`03`	ABS / ESP	Anti-lock brake and stability control
`08`	HVAC (Climatronic)	Air conditioning / automatic climate control
`09`	Central electrics	Body control module / central electronics
`15`	Airbags	Supplemental restraint / airbag control
`16`	Steering wheel	Multifunction steering-wheel module
`17`	Instruments	Instrument cluster
`19`	Gateway	CAN gateway (on CAN-era platforms) / diagnostic gateway
`22`	AWD / Haldex	All-wheel-drive controller
`25`	Immobiliser	Immo box / Kessy on newer platforms
`35`	Central locking	Door and locking electronics
`37`	Navigation	Nav / RNS unit
`46`	Comfort	Central convenience / comfort system
`55`	Headlight range	Automatic headlight levelling
`56`	Radio	Radio / head unit
`76`	Parking aid	Park distance control

KW1281 — the block protocol

KW1281 is best understood by contrast with ISO frames. Where ISO 9141 and KWP2000 are packet protocols (the tester sends a complete packet, the ECU sends a complete packet, turnaround is governed by P-timing), KW1281 is a byte-by-byte handshake protocol: every byte transmitted by either side is echoed by the other side, bit-inverted, before the next byte is sent. The resulting bus trace looks like an interleaved ping-pong rather than two clean frames.

Block structure

len is the total block length including the length byte itself through the `0x03` end byte. **ctr** is a block counter that increments by one on every new block from either side, wrapping modulo 256; testers and ECUs keep their counters in sync or the ECU drops the session. **title** is a single-byte block type (see below). **data** is 0 or more bytes of payload. The block ends with literal `0x03`. Every byte except the final `0x03` must be echoed by the receiver with its bits inverted (i.e. `byte XOR 0xFF`). There is no checksum because the per-byte echo catches any single-bit error immediately.

Common block titles (service bytes)

BYTE	TITLE	MEANING
`0x00`	ACK / no further output	Used to confirm receipt or terminate a sequence
`0x01`	Request measured value group	Payload: group number 01..FF
`0x02`	Request ECU identification	ECU replies with 0xF6 ASCII blocks
`0x03`	End block marker	End of block (not a title byte; used only as terminator)
`0x04`	Request stored / adaptation value	Payload: channel number
`0x05`	Clear DTCs	Acknowledged with 0x09
`0x06`	End output / stop communication	Ends the session
`0x07`	Request DTCs	ECU replies with 0xFC blocks for each DTC
`0x08`	Output test / actuator test	Steps through actuators one by one
`0x09`	ACK with confirmation	Response to a successful command
`0x0A`	NAK / not acknowledged	Response to a malformed command
`0x0F`	Login	Payload: 4-byte login code or similar
`0xF6`	ASCII data block	Payload: printable ASCII string
`0xFC`	DTC report block	Payload: 3-byte DTC record (code, code, status)
`0xFD`	Measured value data block	Payload: measured-value groups of 4 bytes each

KW1281 initialization

Slow init at 5 baud, using the controller number as the address byte. The ECU responds with 0x55, KB1 = `0x01`, KB2 = `0x8A`. Immediately after the standard inverted-KB2 / inverted-address handshake, the ECU spontaneously begins sending `0xF6` ASCII identification blocks: part number, coding, workshop code, serial number, and so on. The tester acknowledges each with `0x09` (ACK) until the ECU sends `0x09` itself, indicating end-of-ident and readiness for commands.

VAG on KWP2000 (KWP6000 internally)

From roughly 1998 for petrol and 2000 for diesel, VAG ECUs added KWP2000 support alongside KW1281. The tester can choose which protocol to run by selecting which 5-baud address byte to send — the same controller numbers work for both — and the ECU's keyword bytes declare which protocol it is speaking. VAG KWP2000 uses service `0x21` ReadDataByLocalIdentifier heavily, with local identifiers that mirror the KW1281 "measured value group" numbers one-for-one. This is deliberate: it let VAG's scan tool (VAS 5051 / 5052) present the same data catalogue regardless of which protocol the ECU supports.

Example — reading measuring-value group 002 on a KWP2000 engine ECU

```
# Session upgrade to extended diagnostic session
82 01 F1 10 86 AA           # 0x10 StartDiagnosticSession, 0x86 extended
83 F1 01 50 86 EF 8F 4A     # positive response

# ReadDataByLocalIdentifier, group 02 = engine basic operating data
83 01 F1 21 02 01 99        # SID 0x21, LID 0x02, record 01
88 F1 01 61 02 01 0A 14 32 ...
   ^^ ^^ ^^ ^^ ^^ ^^ \_______ four data bytes (rpm, load, speed, coolant)
   |  |  |  |  |  +-- record id echoed
   |  |  |  |  +----- LID echoed
   |  |  |  +-------- SID response (0x21 + 0x40)
   |  |  +----------- source (ECU controller 01)
   |  +-------------- target (tester)
   +----------------- format byte: physical, length 8
```

Gotchas

On KW1281, the block counter must start at `0x01` for the first block after init and increment on every subsequent block from either side. A desync aborts the session. When retrying after a missed echo, do not increment the counter — resend the same block.

VAG's KWP2000 implementation frequently returns `0x78` response-pending several times in a row during adaptation writes and flash operations. Implementations that treat a single 0x78 as a transient error and bail out after one will fail on any VAG flash; arm a long timeout (10 s is typical) and keep reading.

Controller `0x19` Gateway on CAN-era VAG cars routes diagnostic traffic to downstream modules over CAN. On a pre-CAN car the Gateway address is not populated; on a post-CAN car selecting controller 01 via K-line is often still possible for legacy tool compatibility, but the actual diagnostic traffic is tunnelled by the gateway.

SECTION 9

OEM profile — BMW DS2 and KWP on E36/E39/E46 era.

BMW's pre-CAN diagnostic stack on E36, E38, E39, E46, E53, and early E60/E83 platforms is called DS2. It is K-line with a BMW-specific framing, module map, and service set. DS2 coexists with ISO 9141-2 OBD-II on the same physical cable but the BMW-specific modules (IKE instrument cluster, LCM light module, SRS, climatronic, radio, …) are only reachable through DS2, not through OBD-II. Post-2003 BMWs migrated enhanced diagnostics to K-DCAN (K-line-CAN bridge) and then pure CAN-UDS, but the DS2 module byte assignments survived in BMW's INPA and EDIABAS toolchain virtually unchanged.

Frame format

DS2 frames are similar in shape to ISO 9141 but with their own conventions:

target	len	data...	CS

target is the module's DS2 address on request or the responding module's address on response. **len** is the total frame length including the target byte through the checksum. **data** is a service byte followed by 0 or more argument bytes. The checksum is the *XOR* of all preceding bytes — different from ISO 9141-2's arithmetic sum; a common source of integration bugs for tools switching between BMW and other OEMs.

Initialization

DS2 uses 5-baud slow init with the module's DS2 address as the address byte. Keyword bytes are typically `0xEF 0x8F`. After the standard KB2 / address inversion handshake the tester can begin sending frames immediately. There is no explicit "StartCommunication" service in DS2; the init itself opens the session.

Module address map

The following are the most commonly used DS2 addresses. A complete list runs to around 80 modules, many of which are present only on specific E-series platforms; the ones below are the ones implementers are most likely to encounter.

ADDR	MODULE	NOTES
`0x00`	Broadcast	Functional broadcast to all modules
`0x12`	DME	Digital motor electronics (engine ECU, Motronic or Bosch ME/MS families)
`0x22`	EGS (older)	Automatic transmission on pre-E46 5-speed platforms
`0x32`	EGS	Automatic transmission (A5S 325Z, A5S 360R, etc.)
`0x34`	ABS / ASC / DSC	Brake and stability control. Exact name varies by year.
`0x44`	EWS	Electronic drive-away protection (immobiliser)
`0x50`	MFL	Multifunction steering wheel
`0x56`	SRS / MRS	Airbag / supplemental restraint
`0x60`	PDC	Park distance control

0x64	IHKA / IHKR	Automatic / manual climate control
0x68	Radio	Business radio / Professional radio head unit
0x6A	CD / CDC	CD changer (trunk-mounted on E39, glovebox on E46)
0x70	RDC	Tyre pressure monitoring (on equipped E39/E46)
0x76	CDC (alt)	Some later CD changer variants
0x80	IKE / Kombi	Instrument cluster
0x9B	NAV / MK3	Navigation / video module
0xA0	TV module	TV tuner (TV-option cars)
0xA4	ZKE / GM	General module / central body electronics
0xA6	Kombi (alt)	Some late E39 / E46 cluster variants
0xB0	SZM	Central switch / centre console
0xC0	RLS	Rain / light sensor
0xC8	TV (alt)	TV on M3 CSL and some Alpinas
0xD0	LCM / LKM	Light control module. Handles headlamps, AHL, fog.
0xE0	AHL	Adaptive headlight / cornering headlamp
0xF0	Tester / broadcast reply	Factory test address

Service bytes

DS2 uses its own service IDs, not the KWP2000 catalogue. The commonly encountered services are:

SID	NAME	NOTES
0x00	Module status / ping	Module responds with a single-byte 0x00 echo if alive
0x01	Read identification	Returns part number, coding, hardware/software version
0x02	Read DTCs	Returns stored fault codes
0x03	Reset adaptations	Clears learned values
0x04	Read memory by address	Payload: 3-byte address, 1-byte length
0x05	Clear DTCs	Clears fault memory
0x06	Read analog input	Platform-specific
0x0B	Write coding	Applies a coding dataset
0x0C	Actuator test	Payload: actuator ID, parameters
0x1D	Component identification	ETL-assigned hardware number
0xA0	Diagnostic session start	Opens an extended session on some modules

Pin-bridging requirement on aftermarket cables

On most E36 and many E39/E46 cars, BMW's DS2 K-line is wired to OBD-II pin 8, not pin 7. BMW's factory tool used the 20-pin round connector under the hood where DS2 was on a dedicated pin. Aftermarket OBD-II tools for BMW therefore require pin 7 and pin 8 to be bridged at the OBD connector. The universally available "INPA K+DCAN" cable has this bridge built into the plug. If you are using a generic

ELM327 or generic K-line adapter, you must add the bridge externally or you will only see emissions traffic and nothing on DS2.

Example — reading DTCs from IKE (instrument cluster)

```
# Request: target 0x80 (IKE), len 3, service 0x02 (read DTCs)
80 03 02 81
   80 = target IKE
   03 = length (target + len + service = 3 bytes before checksum)
   02 = service read DTCs
   81 = checksum = 0x80 XOR 0x03 XOR 0x02

# Response (no faults):
80 04 A0 FF DB
   80 = source IKE (echoed as if it were a target-style header)
   04 = length
   A0 = positive response to 0x02 (BMW adds 0xA0, not 0x40)
   FF = "no DTCs stored" sentinel
   DB = XOR checksum
```

SECTION 10

OEM profile — Subaru SSM2.

Subaru's Select Monitor (SSM) protocol is the cleanest publicly-documented byte-addressed K-line implementation in this reference. SSM1 (pre-1996) ran at 1953 bps on a proprietary 9-pin connector and is obsolete. SSM2 (1996 onward) runs at 4800 bps on some ECUs and 10 400 bps on most post-2002 ECUs; it uses the standard J1962 connector on pin 7. SSM2 is the protocol spoken by Subaru Select Monitor III and by every aftermarket Subaru tuning tool (RomRaider, EcuFlash, Tactrix OpenPort).

Frame format

0x80	dst	src	len	cmd + data ...	CS

0x80 is a fixed start-of-frame byte, present on every SSM2 packet. **dst** is the destination address and **src** is the source address. **len** is the length of the data field in bytes (command byte + arguments), not including the start byte, addresses, length byte, or checksum. **CS** is the 8-bit arithmetic sum of every byte in the frame from the start byte through the last data byte, modulo 256.

Addresses

ADDR	MODULE	NOTES
0x10	ECM	Engine control module. Always present.
0x18	TCM / DCCD	Automatic transmission on 4EAT/5EAT cars; Driver-Controlled Centre Differential on STI with 6MT.
0x20	ABS / VDC	Anti-lock brake / vehicle dynamics control
0x28	Airbag	Supplemental restraint system
0x30	Body / integrated	Body integrated unit (BIU) on newer models
0xF0	Tester	Diagnostic tool source address

Commands

SSM2 command bytes come in request/response pairs: request in the 0xA0–0xBF range, response in the 0xE0–0xFF range. The request command ORed with 0x40 produces the corresponding response command (so 0xA0 → 0xE0, 0xA8 → 0xE8, etc.), analogous to the KWP2000 +0x40 convention.

REQ	RESP	MEANING	NOTES
0xA0	0xE0	ECU init / capability query	Response carries SSM ID (3 bytes), ROM ID (5 bytes), supported-PID bitmask (up to 48 bytes)
0xA8	0xE8	Read address(es)	Request: 3-byte address(es) to read. Can read multiple addresses in a single request.
0xA9	0xE9	Read block	Request: starting address + count. Response: consecutive bytes.
0xAA	0xEA	Write byte(s) by address	Commonly used for real-time table tuning during live ECU logging.

0xB0	0xF0	Write block	Block write to a contiguous address range.
0xB8	0xF8	Address-read with pad mode	Same as 0xA8 but with alternate response packing.

Worked example — ECU init

```
# Request: tester 0xF0 -> ECM 0x10, command 0xA0 (init)
80 10 F0 01 A0 21
   80 = start byte
   10 = destination (ECM)
   F0 = source (tester)
   01 = data length (just the one command byte)
   A0 = command: init
   21 = checksum = (0x80+0x10+0xF0+0x01+0xA0) mod 256

# Response (abridged):
80 F0 10 39 E0 AA BB CC  DD EE FF GG HH ...  CS
   80           = start byte
   F0           = destination (tester)
   10           = source (ECM)
   39           = data length = 0x39 = 57 bytes of payload
   E0           = response: init reply
   AA BB CC     = SSM ID (3 bytes)
   DD EE FF GG HH = ROM ID (5 bytes)
   remaining    = supported-PID bitmask, 48 bytes
   CS           = 8-bit sum checksum
```

The supported-PID bitmask is how a Subaru tool knows which live-data parameters this particular ECU supports, without needing a per-ROM table. Each bit in the 48-byte bitmask corresponds to one documented SSM2 parameter; RomRaider's XML definitions map bit positions to parameter names, units, and address triples.

Why SSM2 is easy to implement against

Unlike most OEM profiles, SSM2 has no session concept, no security access, no response-pending, no timing parameters to negotiate, and a constant frame format. An implementer writes one framer, one checksum function, one address-list constructor, and one response parser. The ECU is responsive for as long as the ignition is on. This simplicity is why the Subaru community has the richest open tooling ecosystem of any OEM: the barriers to entry are several orders of magnitude lower than BMW DS2 or VAG KWP2000.

SECTION 11

OEM profile — Mercedes-Benz 38-pin era and KWP2000.

Mercedes-Benz used a 38-pin round under-hood diagnostic connector on nearly every vehicle sold between about 1987 and 2000, with overlap into the early W203 / W220 era. Where other OEMs route all modules through a single K-line, Mercedes routed each module or module family onto its own dedicated K-line pair on the 38-pin connector. A diagnostic tool selects a module by connecting to the appropriate pins rather than by addressing a byte on a shared bus. The consequence is that on pre-2000 Mercedes there is no single "K-line address map"; there is a connector-pin map, and the byte-level protocol on each pin is negotiated separately. The OBD-II 16-pin connector, where fitted alongside, exposes only the emissions (engine) ECU via standard ISO 9141 / KWP2000.

The 38-pin diagnostic connector

The 38-pin connector is a round, keyed connector typically mounted in the engine bay near the brake booster or fusebox. Mercedes' own tool (HHT, then Star Diagnosis) uses a cable that pig-tails the 38-pin round to the tool's host interface. The most commonly-encountered pin assignments are:

PIN	FUNCTION	NOTES
1	Battery positive (VBAT)	Permanent 12 V
2	Battery positive (switched)	On some variants
3	Engine diagnosis K-line	ME / PMS engine ECU
4	Engine diagnosis L-line	Engine init on L-line for older ECUs
5	Engine diagnosis K-line (alternate)	Second engine line on twin-ECU V8/V12 cars
6	ABS / ESP / BAS K-line	Brake system
7	Ground	Signal and chassis return
8	Shield / ground	Cable shield
10	Transmission K-line	5-speed / 7G-tronic TCU
12	SRS / airbag K-line	Restraint system
13	AAM / central locking K-line	Anti-theft / central locking
14	Instrument cluster K-line	IC / Kombi
15	Instrument cluster L-line	Cluster init
16	HVAC K-line	Automatic climate control
17	ABR (roll-over / headlamp levelling)	Where fitted
18	EA / EDC K-line	Diesel elektronischer Dieselregler
19	ISO 9141 diagnosis (OBD emissions)	Mirrors OBD-II pin 7 where a 16-pin connector is also present
30	SRS / airbag L-line	Airbag init
31	PSE / PZE K-line	Pneumatic central locking pump unit

The above is representative, not exhaustive. Mercedes revised the 38-pin pinout over the years and specific pin assignments vary by chassis (W124, W140, W202, W210, W220). For a production tool, derive the pinout from the Mercedes workshop documentation for the specific chassis rather than from a single reference table.

Protocol variants per pin

Each K-line on the 38-pin connector runs its own protocol. The engine-diagnosis lines (pins 3 / 4 / 5) use KWP2000 with keyword bytes `0xEF 0x8F`. Older petrol engine ECUs (HFM, LH, KE) use Mercedes-specific pre-KWP block protocols sometimes called MBE or HHT-protocol, which are closer in spirit to KW1281 than to ISO 9141. ABS (pin 6), transmission (pin 10), and airbag (pin 12) speak KWP2000 on later cars and simpler block protocols on earlier ones. HVAC (pin 16) and instrument cluster (pin 14) often use slower, more primitive protocols because the modules are older hardware.

W203 / W211 transition

From the W203 C-class (2000) and W211 E-class (2002) onward, Mercedes migrated to a single 16-pin OBD-II connector plus internal CAN. The 38-pin connector was discontinued on most passenger cars by 2004 (commercial vehicles somewhat later). On CAN-era Mercedes, the K-line retains only its OBD-emissions role; all enhanced diagnostics move to CAN with UDS. Star Diagnosis and aftermarket XENTRY-compatible tools remain the canonical way to access the vehicle.

Implementation note for aftermarket tools

On a pre-2000 Mercedes, a tool that connects only to the 16-pin OBD-II connector will see engine-ECU emissions data and nothing else. To reach the cluster, ABS, airbag, transmission, HVAC, and central locking, a 38-to-16 adapter with selectable pin routing is required. Several aftermarket adapter-cable vendors manufacture these; the open-source EDIABAS-compatible community has published wiring diagrams. For a software implementation, the profile key must include the connector type (16-pin OBD-II vs 38-pin round) as well as chassis and model year.

SECTION 12

OEM profile — Nissan Consult and Consult-II.

Nissan Consult is the brand-name for Nissan's diagnostic protocol family. There have been three generations. Consult-I (pre-2000, 14-pin round connector under dash, 9600 bps K-line-like single-wire), Consult-II (2000 onward, standard 16-pin OBD-II connector, 10 400 bps on pin 7), and Consult-III (2007 onward, CAN-based, not covered here). Consult-II is the variant most aftermarket tools target and is what this section describes.

Why discovery is required

Unlike VAG (fixed logical controller numbers), Subaru (fixed byte addresses), or BMW (fixed DS2 byte map), Nissan assigns ECU identifiers by model, region, and platform rather than by a stable cross-platform convention. A Z33 350Z has different ECU IDs from a Y61 Patrol has different IDs from a D22 Navara. The factory tool database resolves ECU IDs at connection time by querying the car. Aftermarket tools (DataScan II, ConZult, NDS-II) mirror this pattern: on first connect, the tool broadcasts a discovery request, each ECU replies with its own ID, and the tool maps IDs to module names by consulting an internal table keyed by VIN fragment or user-selected vehicle.

Module families

Every Nissan vehicle exposes modules from a common set of functional families; the per-vehicle ID assignment is what varies. The families typically present on a post-2000 Nissan are:

FAMILY	NAME	NOTES
ECM	Engine Control Module	Always present. The OBD-II emissions ECU.
TCM	Transmission Control Module	On automatic-transmission cars
ABS	Anti-lock braking system	On ABS-equipped vehicles (virtually all post-2000)
SRS	Supplemental Restraint (airbag)	Always present
BCM	Body Control Module	Locks, lights, wiper logic
IPDM	Intelligent Power Distribution Module	On newer platforms; fuse/relay box with logic
AV	Audio Visual unit	Head unit / navigation
METER	Instrument cluster	Some platforms combine this into BCM
HVAC	Automatic air conditioning	On equipped cars
EPS	Electric power steering	Later platforms (Z33, R35, etc.)

Consult-II frame

Consult-II frames resemble SSM2 in shape but use different service bytes. A request is a 2-byte command (main + sub), optionally followed by arguments. The response is an acknowledgement byte (0xA5 for an error or 0xF0 for OK on some platforms) followed by the requested data. There is no checksum on many Consult-II exchanges; reliability is assumed at the UART level. Services include read live data by parameter ID, read DTCs, clear DTCs, actuator tests, and ECU identification.

Practical implementation advice

For a cross-Nissan tool, the implementation must (a) run a discovery phase on connection, (b) maintain a per-platform table mapping discovered ECU IDs to module-family names, and (c) hold per-family parameter dictionaries because the Consult-II live-data catalogue differs between model families even when the module family is the same. Treating "Nissan" as a single profile is the mistake most aftermarket tools make on their first release; every mature Nissan tool ships a platform selector as its first UI screen.

SECTION 13

Other OEMs — Toyota, Honda, Hyundai/Kia, PSA, Renault, Opel, Volvo, and others.

The remaining volume manufacturers used K-line variants during their post-1996 pre-CAN era, but publicly available byte-level documentation is thin. This section catalogues the known protocol family for each, the approximate era of K-line use, and the implementation approach that has proven productive for aftermarket tooling. Where a public address map exists at all it is noted; where it does not, discovery or per-platform profile is the recommended approach.

Toyota / Lexus

Toyota used ISO 9141-2 for OBD-II emissions from 1996. Enhanced diagnostics on pre-2008 Toyotas used a combination of SAE J1850 VPW (North American Lexus GS and LS 1998-2000), ISO 9141-2 block transfer, and eventually KWP2000 on K-line. The Toyota Tech Stream tool (TIS Techstream) supports all of these. Enhanced addressing is physical-byte based but the byte values are not publicly standardised across platforms; each ECU family (2JZ, 1JZ, 1UR, 2GR, …) has its own conventions. Open-source Toyota diagnostic libraries (OpenTechstream, ToyotaDiag) ship per-platform tables derived from reverse-engineering. On a typical JZX100 Chaser, the engine ECU replies from `0x10` or `0x11`; a NZE121 Corolla may reply from `0x13`. Do not generalise.

Honda / Acura

Honda used ISO 9141-2 until about 2003, then transitioned to K-line KWP2000 on some platforms and directly to CAN on others. The factory tool (HDS) supports both. Honda's K-line enhanced diagnostics are notable for a tight service set that maps cleanly onto KWP2000 but with Honda-specific local identifiers and routine numbers. Honda's immobiliser (IMMO) module deserves a specific warning: security access on Honda IMMOs uses a seed-and-key that has been the subject of several published cryptanalysis papers; open tools can unlock the data sessions but flash-write remains locked.

Hyundai / Kia

Hyundai and Kia used ISO 9141-2 then migrated to KWP2000 on K-line in the mid-2000s before going to CAN around 2010. Enhanced diagnostic addressing uses physical target bytes; a Hyundai engine ECU commonly sits at `0x10`, a TCM at `0x18`, an ABS at `0x28`. The Hyundai factory tool (GDS / GDS-Mobile) is the authoritative source; the aftermarket EASE Hyundai tool and several OpenPort-based alternatives ship per-platform tables.

Mitsubishi

Mitsubishi used ISO 9141-2 with a Mitsubishi-specific keyword-byte variant on many Evo, 3000GT, and truck platforms. Enhanced diagnostics on Evo VII-IX use a K-line stack readable by the OpenPort cable and EvoScan tool. The engine ECU typically responds at `0x10`. Truck platforms (Pajero, L200) shifted to KWP2000 on K-line before CAN.

PSA (Peugeot / Citroën)

PSA is a heavy KWP2000-on-K-line user. From the late 1990s through about 2005, PSA vehicles used KWP2000 fast init with enhanced services above the ISO catalogue for body, BSI (built-in systems interface), and comfort modules. The BSI module acts as a gateway to the rest of the vehicle, similar in spirit to the VAG 19 Gateway but over K-line. Aftermarket support is anchored by PP2000, Lexia, and Diagbox. Addresses are physical-byte but platform-specific; expect a separate profile per platform generation (X, C, F, L on Citroën; T, W on Peugeot).

Renault

Renault used ISO 9141-2 for emissions and KWP2000 on K-line for enhanced diagnostics until Clio IV / Mégane III moved to CAN. The UCH / BCM module on Renault is the central body computer and is the entry point for most comfort-domain diagnostics. CLIP and Can Clip are the factory tools. Aftermarket support is reasonable via Renolink and DDT4ALL.

Opel / Vauxhall

Opel used a Class 2 / SAE J1850 VPW variant on some models and KWP2000 on K-line on others, with broad platform-by-platform variation. The Tech2 tool and its aftermarket clones remain the pragmatic choice for enhanced Opel diagnostics. From Astra H and Zafira B onwards the bus moved to CAN.

Volvo

Volvo ran KWP2000 on K-line on most P80 and P2 platform vehicles (850, S/V70, S/V40, S80, XC90 early) from about 1998 to 2005. Volvo's enhanced diagnostics use a physical-byte addressing scheme with module addresses documented in community-driven VIDA databases. VIDA (Volvo Information and Diagnostic Application) is the factory tool; OpenVIDA and DiCE+VIDA are common aftermarket configurations.

Jaguar / Land Rover

JLR used KWP2000 on K-line across the late-X-type / S-type / XJ8 Jaguar line and Discovery 2 / Freelander 1 / Range Rover L322 (early) Land Rovers. SDD and earlier T4 are the factory tools. The aftermarket Nanocom Evolution and various ELM327-based attempts cover the emissions layer; enhanced diagnostics generally require one of the JLR-specific tools or the IDS/SDD clones.

Porsche

Porsche used KWP2000 on K-line on 996, Boxster 986, Cayenne 955, and early 997 platforms. Enhanced diagnostics use Porsche-specific services layered on the KWP2000 framing. PST2, PIWIS, and PIWIS II are the factory tools; aftermarket Durametric and OBDMfg Porsche tools cover a useful subset. Porsche's immobiliser (Anti-Theft Protection) is one of the more difficult security-access targets in consumer automotive.

Fiat / Alfa / Lancia

Fiat Group used KWP2000 with fast init on K-line extensively from about 1999 to 2008 before moving to CAN. Alfa 147, 156, 166, GT and Fiat Punto, Stilo, Bravo, and Lancia Thesis, Y, Lybra are within the K-line era. Addressing is physical-byte; the MultiECUScan tool and Fiat/Alfa-ECU-Scan cover much of the aftermarket use case.

SECTION 14

Implementation guidance — state machines, profile keys, error handling.

This section collects the design recommendations implied by the preceding material into a checklist for building a K-line diagnostic implementation that spans multiple OEMs. It is organised by architectural decision, not by feature: the goal is to avoid the common integration traps rather than to catalogue what a tool can do.

The canonical connection state machine

Every K-line diagnostic session runs through the same high-level state machine regardless of OEM. A robust implementation encodes this explicitly rather than letting it emerge from imperative code.

#	STATE	MEANING
1	`IDLE`	No active session. K-line driver released, UART closed or in passive listen.
2	`INIT_PENDING`	Tester has begun an init sequence (5-baud or fast). Waiting for ECU handshake completion.
3	`SESSION_DEFAULT`	Init succeeded. ECU is in default / standard session. OBD-style reads permitted.
4	`SESSION_EXTENDED`	Tester has issued StartDiagnosticSession with an extended sub-function and received a positive response. Richer service set enabled.
5	`SESSION_PROGRAMMING`	Flash reprogramming session. Use only during actual flashing operations.
6	`SECURITY_UNLOCKED`	SecurityAccess (0x27) has completed successfully within the current session. Typically required for writes.
7	`REQUEST_IN_FLIGHT`	Tester has transmitted a request and is waiting for a response or a 0x78 pending. P2 timer armed.
8	`RESPONSE_PENDING`	ECU has returned at least one 0x78. Re-arm P2* timer on each 0x78 received. Do not abort.
9	`KEEPALIVE`	Background state. TesterPresent fires every P3/2 (typically 2 s) whenever no other traffic is outgoing.
10	`ERROR_RECOVERABLE`	Checksum mismatch, unexpected NRC, or P2 timeout. Retry up to 3 times then fall back to SESSION_DEFAULT or IDLE.
11	`ERROR_FATAL`	Repeated timeouts, bus-off-equivalent condition, or explicit StopCommunication from ECU. Drop to IDLE; user must re-connect.

Profile-key design

A cross-OEM tool must select the correct protocol profile before it can sensibly transmit anything. Profile selection is not brand-granular; it is platform-and-module-granular. A profile key that supports real-world vehicles has at least the following dimensions:

DIMENSION	PURPOSE
Make	Volkswagen, BMW, Subaru, … — coarse bucket

Platform / chassis	B6, E46, GD, W203, … — fine-grained; determines wiring, init, and addresses
Model year	Within a platform, behaviours often change between early and late production
Market	US, EU, JDM — ECU calibrations and emissions-required services differ
Module family	Engine, transmission, ABS, … — each module has its own profile even within the same vehicle
Connector type	16-pin OBD-II, 20-pin BMW round, 38-pin Mercedes round, 14-pin Nissan Consult — determines physical routing before any byte is transmitted
Protocol variant	ISO 9141-2 vs KWP2000 vs KW1281 vs DS2 vs SSM2 vs Consult-II — derived, not user-selected
Init strategy	5-baud slow init (with specific address byte) vs fast init — derived from protocol variant
Addressing model	Functional (0x33), physical byte, logical controller number, or connector-pin selection
Timing profile	P2 max, P3 max, inter-byte gap — per-profile table

Error-handling patterns

The 0x78 loop

The single most common reason a K-line tool fails on first deployment to real vehicles is mis-handling `0x78`. An ECU performing a routine (flash operation, EEPROM write, adaptation reset) may return `0x78` every few hundred milliseconds for ten or more seconds. A tester that treats the first `0x78` as an error and retries the request from scratch will collide with the still-pending operation. The correct behaviour: re-arm P2* timer on each `0x78`, do not transmit anything until either the final positive / non-0x78 negative response arrives or P2* expires. Bound the total pending time with a profile-specific absolute ceiling (30 s is typical for flash) to prevent runaway.

Checksum and framing errors

Three common failure modes, all recoverable: (a) the tester computes the wrong checksum and the ECU silently drops the frame, giving a P2 timeout; (b) the ECU returns a frame the tester's framer cannot parse (common on BMW DS2 when the XOR checksum was mistaken for a sum checksum); (c) a mid-frame gap exceeds P1, causing the tester to abort mid-reception and miss the next request's echo. Log every abort with the raw bytes received. Without the raw log, debugging these is guesswork.

Session loss during long operations

TesterPresent must fire during every slow operation. A flash write that takes 45 s will lose the extended session if keep-alive is paused. The cleanest implementation runs a background timer that transmits `0x3E 0x01` (TesterPresent with suppress-positive-response) exactly once per 2 s whenever no other outgoing traffic is queued. Note: do not fire TesterPresent while a request is in flight or awaiting a 0x78 loop — it will collide with the ECU's response.

Retry logic

A reasonable retry strategy: on no response within P2 max, retry the request twice with a 100 ms back-off between attempts. On three consecutive failures, drop to SESSION_DEFAULT and re-attempt the session upgrade; on three consecutive session-upgrade failures, drop to IDLE and restart init. On the final failure, surface the raw response bytes to the user. Never silently absorb unexpected NRCs; report them.

Logging & reproducibility

A production diagnostic tool should log every byte transmitted and received with microsecond timestamps relative to init. This enables post-hoc diagnosis of timing violations, ECU-specific quirks, and adapter issues. The log format should be plain text (line per byte or line per frame) rather than a binary format, because the logs will be exchanged with ECU vendors, vehicle owners, and community members who do not have the tool vendor's parser. Two conventions are established: the EDIABAS "trace file" (ifh.trc) format used by BMW-focused tools, and the PuTTY-log style raw byte capture used elsewhere. Either is fine; consistency matters more than choice.

APPENDIX A

Complete KWP2000 service-ID reference.

Positive response in every case is the SID plus 0x40. Negative response is `0x7F` followed by the original SID followed by an NRC (Appendix B).

SID	NAME	GROUP	USE
`0x10`	`StartDiagnosticSession`	Diagnostic management	Enter a diagnostic session; sub-function selects mode
`0x11`	`ECUReset`	Diagnostic management	Soft- or hard-reset the ECU
`0x14`	`ClearDiagnosticInformation`	Stored data	Clear fault memory; argument selects DTC group
`0x17`	`ReadStatusOfDiagnosticTroubleCodes`	Stored data	Read status mask for DTCs
`0x18`	`ReadDiagnosticTroubleCodesByStatus`	Stored data	Read DTCs filtered by status; commonly used for "read stored faults"
`0x1A`	`ReadECUIdentification`	Stored data	Read ECU part number, serial, hardware / software version
`0x20`	`StopDiagnosticSession`	Diagnostic management	Leave the current session; ECU returns to default
`0x21`	`ReadDataByLocalIdentifier`	Data transmission	Read a locally-defined data record. VAG's standard measured-value interface.
`0x22`	`ReadDataByCommonIdentifier`	Data transmission	Read a 2-byte-identifier data record. The OBD-II-era successor to 0x21.
`0x23`	`ReadMemoryByAddress`	Data transmission	Read N bytes starting at a given address
`0x25`	`StopRepeatedDataTransmission`	Data transmission	Cancel a periodic data stream started by 0x2A
`0x26`	`SetDataRates`	Data transmission	Configure periodic data-transmission rate
`0x27`	`SecurityAccess`	Upload / download	Seed-and-key handshake to unlock restricted services
`0x28`	`DisableNormalMessageTransmission`	Communication control	Silence normal functional messages during diagnostics
`0x29`	`EnableNormalMessageTransmission`	Communication control	Re-enable normal functional messages
`0x2A`	`ReadDataByPeriodicIdentifier`	Data transmission	Start a periodic data stream
`0x2C`	`DynamicallyDefineLocalIdentifier`	Data transmission	Compose a local identifier from multiple memory / DID sources
`0x2E`	`WriteDataByCommonIdentifier`	Data transmission	Write a 2-byte-identifier data record. Used for coding on many post-2005 OEMs.
`0x2F`	`InputOutputControlByCommonIdentifier`	I/O control	Actuator tests, forced I/O states, keyed by common identifier
`0x30`	`InputOutputControlByLocalIdentifier`	I/O control	Actuator tests keyed by local identifier
`0x31`	`StartRoutineByLocalIdentifier`	Remote activation	Start an ECU-side routine (e.g. throttle-body learn)
`0x32`	`StopRoutineByLocalIdentifier`	Remote activation	Request a running routine to stop

0x33	RequestRoutineResultsByLocalIdentifier	Remote activation	Poll a routine's current status or final result
0x34	RequestDownload	Upload / download	Begin a flash write operation; arguments specify address and size
0x35	RequestUpload	Upload / download	Begin a flash read-out operation
0x36	TransferData	Upload / download	Carry a block of data during an active download / upload
0x37	RequestTransferExit	Upload / download	Signal end of download / upload; ECU verifies and finalises
0x38	StartRoutineByAddress	Remote activation	Start a routine identified by absolute address (legacy)
0x39	StopRoutineByAddress	Remote activation	Stop a routine identified by address
0x3A	RequestRoutineResultsByAddress	Remote activation	Poll results from an address-identified routine
0x3B	WriteDataByLocalIdentifier	Data transmission	Write a local-identifier record. VAG adaptation channel writes go here.
0x3D	WriteMemoryByAddress	Data transmission	Write N bytes to a given address. Protected by SecurityAccess.
0x3E	TesterPresent	Diagnostic management	Keep-alive. Sub-function 0x01 suppresses the positive response.
0x7F	NegativeResponse (not a service)	—	Appears as the first data byte of every negative response
0x81	StartCommunication	Diagnostic management	Open the data link after fast init. ECU replies with keyword bytes.
0x82	StopCommunication	Diagnostic management	Close the data link cleanly
0x83	AccessTimingParameters	Diagnostic management	Read or set P-timing parameters, including P2*
0x84	NetworkConfiguration	Diagnostic management	Configure logical channels (rarely used on K-line)
0x85	ControlDTCSetting	Diagnostic management	Suspend or resume DTC storage during invasive tests
0x86	ResponseOnEvent	Diagnostic management	Configure ECU to asynchronously report when an event condition occurs

APPENDIX B

Negative-response-code reference.

Every KWP2000 negative response has the form `0x7F <origSID> <NRC>`. The complete table of NRCs defined in ISO 14230-3, with the most common values prominent, is listed below. Values not listed are either reserved or are OEM-specific extensions.

NRC	NAME	MEANING
`0x10`	`generalReject`	The requested action has been rejected for a non-specific reason
`0x11`	`serviceNotSupported`	The ECU does not implement this service
`0x12`	`subFunctionNotSupported`	The service is implemented but the specific sub-function is not
`0x13`	`incorrectMessageLengthOrInvalidFormat`	Message length or arguments do not match the service specification
`0x21`	`busyRepeatRequest`	ECU is temporarily busy. Repeat the request after a short delay.
`0x22`	`conditionsNotCorrectOrRequestSequenceError`	Preconditions for the service are not met (e.g. wrong session, engine running when it should be off)
`0x23`	`routineNotCompleteOrServiceInProgress`	A previously-started routine is still running
`0x31`	`requestOutOfRange`	A parameter is valid in form but out of permitted range (e.g. unknown local identifier)
`0x33`	`securityAccessDenied`	Service is protected; SecurityAccess has not been unlocked
`0x35`	`invalidKey`	SecurityAccess key does not match the expected value for the supplied seed
`0x36`	`exceededNumberOfAttempts`	Too many failed SecurityAccess attempts; ECU is temporarily locked
`0x37`	`requiredTimeDelayNotExpired`	SecurityAccess retry is still within the penalty delay window (typically 10 s)
`0x40`	`downloadNotAccepted`	RequestDownload rejected (wrong session, security locked)
`0x41`	`improperDownloadType`	Download format code not supported by ECU
`0x42`	`cannotDownloadToSpecifiedAddress`	Target address is outside permitted flash region
`0x43`	`cannotDownloadNumberOfBytesRequested`	Requested download size is too large for ECU buffer or region
`0x50`	`uploadNotAccepted`	RequestUpload rejected
`0x51`	`improperUploadType`	Upload format code not supported
`0x52`	`cannotUploadFromSpecifiedAddress`	Source address not readable
`0x53`	`cannotUploadNumberOfBytesRequested`	Requested upload size rejected
`0x71`	`transferSuspended`	TransferData interrupted
`0x72`	`transferAborted`	TransferData terminated by ECU due to error
`0x74`	`illegalAddressInBlockTransfer`	Address outside permitted region during block transfer

0x75	illegalByteCountInBlockTransfer	Block size out of range
0x76	illegalBlockTransferType	Block transfer type code not supported
0x77	blockTransferDataChecksumError	Checksum of a transferred block does not match
0x78	requestCorrectlyReceivedResponsePending	ECU acknowledges request, is still working. Re-arm P2*, do not retry.
0x79	incorrectByteCountDuringBlockTransfer	Transferred block length does not match declared length
0x80	subFunctionNotSupportedInActiveSession	Sub-function is defined but not permitted in the current session

APPENDIX C

Extended VAG logical-controller reference.

Controller numbers used by VAG workshop tools from KW1281 through KWP2000 through UDS-over-CAN. Numbers below 0x80 (decimal 128) are the original classic set used on K-line; numbers above are extensions introduced as CAN-era modules proliferated.

ADDR	MODULE	DESCRIPTION
01	Engine	Engine control module (petrol or diesel)
02	Transmission	Automatic transmission control module
03	ABS / ESP	Anti-lock brake and stability control
04	Steering angle sensor	Dedicated steering-angle sender
05	Access / start (Kessy)	Keyless access on newer platforms
06	Seat memory, passenger	Passenger-side seat memory
07	Control head / info display	Centre console display on MMI-era cars
08	HVAC / Climatronic	Automatic climate control
09	Central electrics	Body control / main fusebox with logic
0D	Sunroof	Slide / tilt sunroof module
0F	Digital radio (DAB)	DAB tuner
10	Park / steer assist	Park-steering assist ECU
11	Engine II	Second engine ECU on V8 / V10 / V12
13	Adaptive cruise / ACC	ACC long-range radar
15	Airbag	Supplemental restraint module
16	Steering wheel electronics	Multifunction wheel module
17	Instruments	Instrument cluster
18	Auxiliary heating	Webasto / Eberspacher interface
19	Gateway	Diagnostic / CAN gateway
1C	Position sensing	Vehicle-position sensor (air suspension)
1D	Driver identification	Personalisation module
1E	Media player 1	Primary CD / media changer
1F	Sat tuner (SDARS)	XM / Sirius satellite radio
20	High beam assist	Automatic high-beam
21	Engine III	Third engine ECU on some high-cylinder-count
22	AWD (Haldex)	Haldex / quattro ultra all-wheel-drive
23	Brake boost / electromechanical	Electromechanical brake assist
25	Immobiliser	Immo box
26	Auto roof	Convertible folding-roof
28	HVAC, rear	Rear climate zone

29	Left light	Left front headlamp electronics
2B	Steering column lock	Electronic steering-column lock
2D	Intercom	Rear passenger intercom
2E	Media player 2	Secondary media device
30	Sport differential	Torque-vectoring rear diff
31	ACC long range	Alternative ACC ECU on some platforms
34	Level control	Air suspension / ride-height
35	Central locking	Door and locking electronics
36	Seat memory, driver	Driver-side seat memory
37	Navigation	Nav / RNS unit
38	Roof electronics	Panoramic roof logic
39	Right light	Right front headlamp electronics
42	Driver door	Driver-door control module
44	Steering assist	Electric power steering
46	Comfort / convenience	Central convenience system
47	Sound system	Amplifier / active sound
52	Passenger door	Passenger-door control module
53	Parking brake	Electromechanical parking brake
55	Headlight range	Automatic headlight levelling
56	Radio	Radio / head unit (classic)
62	Left rear door	Rear-left door module
65	Tyre pressure	TPMS
67	Voice control	Voice command
69	Trailer function	Towing-module / trailer-detection
72	AUX input	Aux audio gateway
76	Parking aid	PDC
77	Telephone	Bluetooth / telephony
82	Passenger rear door	Rear-right door module
A5	Front sensor cluster	Radar / camera sensor cluster
A9	Structure-borne sound	Active sound / noise cancellation
BC	Front camera	Front-facing camera for lane / sign

APPENDIX D

OBD-II Mode 01 PID reference (selected).

Selected Mode 01 PIDs from SAE J1979 / ISO 15031-5. A, B, C, D denote the first, second, third, and fourth returned data bytes respectively. PID 0x00 is the support bitmap for PIDs 01–20; 0x20 is the support bitmap for 21–40; 0x40 for 41–60; etc. Query PID 0x00 first to learn which PIDs the ECU supports before attempting the rest.

PID	NAME	BYTES	FORMULA / UNIT
0x00	Supported PIDs 01-20	4	Bitmask. Bit N set = PID supported.
0x01	Monitor status since DTC clear	4	MIL state, DTC count, readiness monitors
0x02	Freeze DTC	2	DTC that caused freeze frame, BCD-encoded
0x03	Fuel system status	2	Open/closed-loop flags per bank
0x04	Calculated engine load	1	A * 100 / 255 [%]
0x05	Engine coolant temperature	1	A - 40 [°C]
0x06	Short-term fuel trim, bank 1	1	(A - 128) * 100 / 128 [%]
0x07	Long-term fuel trim, bank 1	1	(A - 128) * 100 / 128 [%]
0x08	Short-term fuel trim, bank 2	1	(A - 128) * 100 / 128 [%]
0x09	Long-term fuel trim, bank 2	1	(A - 128) * 100 / 128 [%]
0x0A	Fuel pressure	1	A * 3 [kPa]
0x0B	Intake manifold abs pressure	1	A [kPa]
0x0C	Engine RPM	2	(A*256 + B) / 4 [rpm]
0x0D	Vehicle speed	1	A [km/h]
0x0E	Timing advance	1	(A / 2) - 64 [° before TDC]
0x0F	Intake air temperature	1	A - 40 [°C]
0x10	MAF air flow rate	2	(A*256 + B) / 100 [g/s]
0x11	Throttle position	1	A * 100 / 255 [%]
0x12	Commanded secondary air status	1	Bit-encoded
0x13	O2 sensors present (8 banks)	1	Bit per sensor
0x14	O2 sensor 1, voltage + STFT	2	A/200 [V], STFT = (B - 128) * 100 / 128 [%]
0x15	O2 sensor 2, voltage + STFT	2	as above
0x1C	OBD standards conformance	1	Enumeration: 01=CARB, 02=EPA, 03=OBD+OBD2, …
0x1F	Runtime since engine start	2	A*256 + B [s]
0x20	Supported PIDs 21-40	4	Bitmask
0x21	Distance with MIL on	2	A*256 + B [km]
0x22	Fuel rail pressure (vac ref)	2	(A*256 + B) * 0.079 [kPa]
0x2F	Fuel tank level	1	A * 100 / 255 [%]
0x30	Warm-ups since DTC clear	1	A [count]
0x31	Distance since DTC clear	2	A*256 + B [km]

0x33	Barometric pressure	1	A [kPa]
0x3C	Catalyst temperature, B1S1	2	((A*256 + B) / 10) - 40 [°C]
0x40	Supported PIDs 41-60	4	Bitmask
0x42	Control module voltage	2	(A*256 + B) / 1000 [V]
0x43	Absolute load value	2	(A*256 + B) * 100 / 255 [%]
0x44	Commanded lambda equiv ratio	2	(A*256 + B) * 2 / 65536
0x45	Relative throttle position	1	A * 100 / 255 [%]
0x46	Ambient air temperature	1	A - 40 [°C]
0x47	Absolute throttle position B	1	A * 100 / 255 [%]
0x4C	Commanded throttle actuator	1	A * 100 / 255 [%]
0x4D	Time run with MIL on	2	A*256 + B [min]
0x4E	Time since DTC clear	2	A*256 + B [min]
0x51	Fuel type	1	Enumeration
0x5C	Engine oil temperature	1	A - 40 [°C]
0x5E	Engine fuel rate	2	(A*256 + B) * 0.05 [L/h]

APPENDIX E

Source and standards list.

Primary standards

STANDARD	TITLE / CONTENT
ISO 9141-2	Road vehicles — Diagnostic systems — CARB requirements for interchange of digital information. Defines K-line physical layer and basic frame format.
ISO 14230-1	Road vehicles — Diagnostic systems — Keyword Protocol 2000 — Part 1: Physical layer. Successor to ISO 9141-2 at the physical layer.
ISO 14230-2	Road vehicles — Diagnostic systems — KWP2000 — Part 2: Data link layer. Defines frame formats, addressing, initialization, timing.
ISO 14230-3	Road vehicles — KWP2000 — Part 3: Application layer. Defines service identifiers, negative response codes, and service semantics.
ISO 14230-4	Road vehicles — KWP2000 — Part 4: Requirements for emission-related systems. Defines the OBD-II-over-KWP2000 bindings used on K-line emissions diagnostics.
SAE J1962	Diagnostic Connector Equivalent to ISO/DIS 15031-3. Defines the 16-pin OBD-II connector pinout.
SAE J1979	E/E Diagnostic Test Modes Equivalent to ISO/DIS 15031-5. Defines the OBD-II modes 01-0A, freeze-frame data structures, and PID catalogue.
SAE J2178	Class B data communication network messages. Companion to J1850 for cross-reference.
ISO 15031-5	Emissions-related diagnostic services. The ISO version of J1979.
ISO 14229 (UDS)	Unified Diagnostic Services. Successor to KWP2000 on CAN; referenced here for services that carry over with unchanged SIDs.

Open tool documentation (implementation reference)

TOOL	WHAT IT DOCUMENTS
VCDS	Ross-Tech diagnostic software. The de-facto reference for VAG controller numbering and KW1281 / KWP6000 block semantics.
EDIABAS / INPA	BMW's internal diagnostic platform, widely documented in the aftermarket community. Primary reference for DS2 module bytes and service semantics.
RomRaider	Open-source Subaru tuning suite. Defines SSM2 framing, command set, and parameter dictionaries in open XML.
EcuFlash / Tactrix OpenPort	Open-source ECU flashing. Reference for SSM2 flash protocol and Mitsubishi Evo K-line.
Star Diagnosis / XENTRY	Mercedes factory tool. Reference for 38-pin pinout conventions and platform-specific protocols.
DataScan II	Nissan aftermarket tool documentation. Reference for Consult-II frame format and module-family conventions.
DiCE / VIDA	Volvo Diagnostic Communication Equipment. Reference for Volvo KWP2000 on K-line module addressing.
PP2000 / Lexia / Diagbox	PSA (Peugeot / Citroën) diagnostic suites. Reference for PSA fast-init K-line conventions.

Tech2 / Tech2Win	Opel / GM diagnostic platform. Reference for Opel K-line and Class 2 conventions.

Community references

COMMUNITY	FOCUS
NEFMoto, VAG-COM community	KW1281 block-protocol reverse-engineering and VAG platform-by-platform addressing
Bimmerforums, E46 Fanatics	BMW DS2 address map, module-specific quirks, pin-7/8 bridging discussion
Romraider wiki	SSM2 supported-PID bitmask format, ROM-ID conventions
Nissan Consult tool forums	Consult-I vs Consult-II protocol differences and platform-specific ECU-ID ranges
MHH Auto, EcuFlash forums	Cross-OEM K-line reverse-engineering notes, security-access seed-key samples

A note on authoritativeness

Where this reference and an ISO / SAE standard disagree, the standard is authoritative. Where this reference and OEM factory documentation disagree, the OEM factory documentation is authoritative for that OEM. Where this reference and open community documentation disagree, apply engineering judgment: community documentation is often more accurate than published OEM material for reverse-engineered behaviours, but is also more likely to contain platform-specific assumptions dressed up as universal claims. When in doubt, capture a bus trace from a known-good factory tool and observe.

— end of reference —